UCOMPENSAR

Ingeniería de Software
Modalidad virtual

Documento técnico de Esto Es Cachapas y Algo Más para Android

Actividad 4
Segmento de profundización

Desarrollo de aplicaciones móviles nativas

Presentado por
Javier Andrés Barrera Cárdenas
Trabajo individual

10 de noviembre de 2026
Colombia

En este documento presento el desarrollo de las interfaces Android de Esto Es Cachapas y Algo Más, teniendo en cuenta el menú del negocio y las funciones de cada rol. Los pagos y la recuperación de contraseña se muestran como una demostración local para revisar el funcionamiento de las pantallas.

Contenido

Sección	Página
Introducción y enfoque del proyecto	3
Pantallas y funciones por rol	4
Código XML y recursos de Android	5
Catálogo y precios del negocio	7
Modelo local y reglas del negocio	8
Interfaz de acceso ejecutada	10
Validación y comprobaciones	11
Ejecución y generación del APK	13
Publicación y materiales de entrega	14
Guion del video técnico	16
Privacidad y adaptación normativa	18
Referencias	19

Introducción y enfoque del proyecto

Lo que propongo con Esto Es Cachapas y Algo Más es una tienda virtual para Android en la que se puedan consultar y comprar los productos del negocio y en este documento organizo el desarrollo de sus interfaces, por ejemplo: cómo se presenta el menú, qué puede hacer cada usuario y cómo se realiza un pedido de prueba. La idea principal aquí es contar con una guía que explique lo que ya se implementó y que también permita revisar lo que falta antes de utilizar la aplicación en un negocio real.

Elegí este enfoque porque me permite trabajar con el menú de mi proyecto de cachapas y teniendo en cuenta que la aplicación conserva el verde y el amarillo del negocio, los precios se presentan en pesos colombianos y el catálogo cuenta con las 29 referencias y presentaciones de la imagen entregada. Para revisar el inventario se asignaron 20 unidades a cada referencia y me parece importante aclarar que este valor es un dato de demostración que se puede editar.

Cambios respecto a TecnoConecta

Aspecto	Documento anterior	Implementación de la actividad 4
Negocio	Accesorios tecnológicos	Comidas del menú de Esto Es Cachapas y Algo Más
Interfaz	Jetpack Compose propuesto	Layouts XML para explicar layouts, drawables, estilos y values
Organización	Varias tiendas y vendedores	Un negocio con perfiles de comprador, vendedor y administrador
Datos y servicios	Firebase y Wompi propuestos	Repositorio local persistente y pasarela simulada

Alcance y límites de la versión

Como primera parte decimos que la aplicación cuenta con catálogo, cuentas, perfiles, carrito y pedidos y adicionalmente presenta una pasarela de pagos de prueba; por su parte, el registro, los CRUD y el historial trabajan con los datos del dispositivo y no se sincronizan entre teléfonos. La recuperación de contraseña muestra un código local y la biometría utiliza el diálogo de Android cuando el equipo dispone de un autenticador compatible, por lo que estas funciones permiten revisar las interfaces sin presentar el prototipo como un servicio comercial ya conectado.

La compra de demostración se maneja con recogida sin cargo y resulta útil mantener este recorrido mientras se confirman la cobertura y la tarifa de domicilio, dado que la aplicación bloquea el pago de esa opción cuando todavía no se tiene la información. La dirección del local, el contacto comercial y los ingredientes que no aparecen en el menú deberán completarse con los datos del negocio.

Pantallas y funciones por rol

Rol	Pantallas implementadas	Comportamiento
Comunes	Splash, ingreso, registro y recuperación	Cuenta nueva como comprador; credenciales locales y acceso biométrico previamente activado.
Comprador	Inicio, menú y detalle	Búsqueda por nombre, filtros por categoría, precio y disponibilidad.
Comprador	Carrito y confirmación de pago	Agregar, consultar, cambiar cantidades, retirar líneas y confirmar una compra simulada.
Comprador	Pedidos, detalle y perfil	Consultar historial propio, cancelar intentos sin confirmar y editar datos personales.
Vendedor	Panel, productos y formulario	Crear, consultar, editar y eliminar mediante baja lógica; control de disponibilidad.
Vendedor	Pedidos y perfil	Consultar pedidos recibidos y avanzar por los estados operativos permitidos.

Rol	Pantallas implementadas	Comportamiento
Administrador	Panel, usuarios y formulario	Listar, crear, editar y desactivar cuentas; asignar roles.
Administrador	Productos, pedidos y reporte	Gestionar el menú, atender pedidos y consultar ventas agregadas por producto.

Recorridos de navegación

Comprador. El recorrido comienza en el ingreso y continúa por inicio, menú, detalle, carrito, confirmación y resultado del pedido y teniendo en cuenta que la barra inferior conserva inicio, menú, carrito y perfil, el usuario puede volver a estas opciones durante la navegación. Desde inicio y perfil también se puede consultar el historial de compras.

Vendedor. Después del ingreso se abre el panel y desde allí se consultan los productos para crear o editar una publicación y adicionalmente se revisan los pedidos recibidos para cambiar su estado. Administrador. El panel permite pasar a usuarios, productos, pedidos y reporte de ventas y los botones de volver mantienen el recorrido dentro del rol autorizado.

Para organizar estas pantallas se utilizaron cinco Activities para el acceso y el contenedor principal y un Fragment con destinos separados mediante argumentos y layouts propios, por lo que el botón de retroceso utiliza la pila de fragmentos. Los formularios con identificadores conservan su estado de vista cuando se recrea la pantalla y el carrito y los pedidos se guardan en el almacenamiento local.

Código XML y recursos de Android

En esta parte utilicé componentes nativos y Material Components y teniendo en cuenta que los archivos XML definen la estructura y el estilo de las pantallas, Kotlin se encarga de presentar los datos, validar las entradas y atender las acciones del usuario. Los colores y las dimensiones se reutilizan entre los roles y me parece una forma manejable de conservar la misma presentación en el aplicativo.

Recurso	Archivos o elementos	Uso
Layouts	res/layout/activity_main.xml	LinearLayout, MaterialToolbar, FragmentContainerView y BottomNavigationView.
Pantallas	res/layout/fragment_*.xml	ScrollView, formularios, Spinner, CheckBox y grupos de acciones.
Componentes	res/layout/item_card.xml	Tarjeta reutilizada para productos, usuarios y pedidos.
Drawables	bg_card.xml, bg_hero.xml, ic_*.xml	Fondos con esquinas, gradiente e iconos vectoriales.
Values	colors.xml, styles.xml, dimens.xml y strings.xml	Paleta, tipografía, tamaños, textos comunes y tema.
Menús	res/menu/nav_buyer.xml, nav_seller.xml y nav_admin.xml	Destinos inferiores disponibles para cada rol.

Decisiones visuales

Para el diseño conservé el verde #28732C en las acciones y los títulos y lo acompañé con el verde oscuro #164C22, mientras que el amarillo #FFCF14 aparece en la marca y el fondo crema #FFFCF3 permite distinguir las tarjetas blancas. Los controles principales tienen una altura mínima de 48 dp y los formularios muestran sus etiquetas y mensajes de validación en español.

Ejemplo para explicar en el video

Por ejemplo, en fragment_catalog.xml el ScrollView contiene la búsqueda y un ChipGroup y después ScreenFragment filtra el catálogo por nombre y categoría e infla item_card.xml para mostrar cada resultado. El precio se presenta con Rules.money y en styles.xml se utiliza PrimaryButton para definir las esquinas, el tamaño del texto y la altura del botón, de manera que un cambio en colors.xml se refleja en las pantallas que utilizan ese recurso.

Catálogo y precios del negocio

El catálogo se encuentra en app/src/main/assets/menu.json y sus nombres y precios iniciales corresponden al menú original del negocio, teniendo en cuenta que la imagen también está incluida en la aplicación para consultar la referencia completa. De esta manera se puede comparar la información de la pantalla con los productos que se ofrecen en el proyecto.

Cachapas	Precio COP
Sencilla	$8.000
Full Queso	$12.000
Con Jamón	$10.000
Con Tocineta	$12.000
Con Carne Mechada	$15.000
Con Pechuga a la Plancha	$15.000
Con Cochino Frito	$25.000
Con Chorizo	$15.000

Categoría y presentaciones	Precio COP
Hamburguesa Especial con tocineta y queso amarillo	$10.000
Hamburguesa Mixta / Trifásica	$16.000 / $22.000
Pizzas de 8 porciones: Chicago, New York, Hawaiana, Pollo Tocineta, Napolitana, Mexicana, Champiñones y La Caprichosa	$25.000 cada una
Shawarma	$20.000
Salchipapas / Salchipollo	$15.000 / $20.000 / $30.000

Categoría y presentaciones	Precio COP
Picadas	$20.000 / $30.000 / $50.000

En los combos se identificó cada presentación por su precio, puesto que el menú no especifica tamaños ni cantidades y no sería adecuado asignarles una porción diferente sin confirmarla. Las descripciones también indican que los ingredientes y los alérgenos deben consultarse con el negocio cuando todavía no se dispone de una ficha detallada.

Modelo local y reglas del negocio

Separación del código

Archivo Kotlin	Responsabilidad
Domain.kt	Modelos Product, User, OrderLine y Order; reglas de precio, cantidad, total y transición.
Store.kt	Persistencia JSON en SharedPreferences, credenciales, permisos, carrito y pedidos.
Activities.kt y MainActivity.kt	Ingreso, registro, recuperación, splash, contenedor y navegación por rol.
ScreenFragment.kt	Inflar layouts, presentar información, validar formularios y ejecutar acciones.
BiometricGate.kt	Cifrado del identificador de sesión con Android Keystore y BiometricPrompt.

Cada usuario tiene su carrito y su historial y antes de confirmar se revisan los productos activos, las cantidades y el inventario, por lo que el pedido guarda una copia del nombre, el precio y la cantidad de cada línea. Me parece una decisión útil porque si después se edita un producto del menú, la venta anterior conserva los datos con los que fue registrada.

Cuando el resultado de prueba es aprobado se descuenta el inventario y se vacía el carrito, mientras que un intento pendiente o rechazado conserva los productos y no reserva unidades. El comprador puede cancelar un intento sin confirmar y teniendo en cuenta que esta versión no concilia pagos externos, un intento pendiente no se convierte por sí solo en una venta aprobada.

Los estados del pedido se organizan en recibido, en preparación, listo para recoger y entregado y el negocio puede cancelar mientras el pedido todavía se encuentra recibido; en ese caso se repone el inventario y se registra un reintegro simulado. El historial se conserva para que la cancelación pueda consultarse después y no desaparezca como si el pedido nunca hubiera existido.

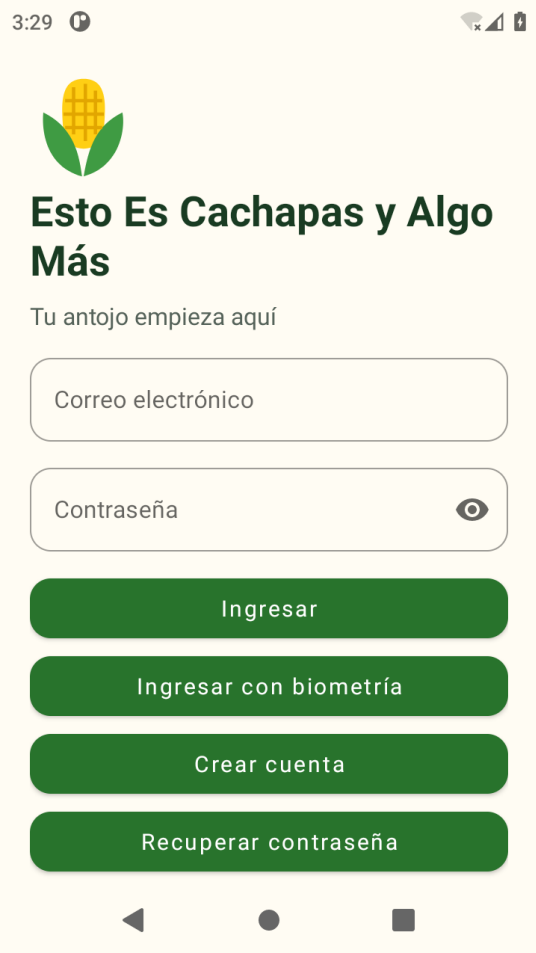
Acceso y persistencia

El repositorio comprueba el rol antes de modificar usuarios o productos y también impide que el administrador quite su propio acceso o que el negocio quede sin un administrador activo, teniendo en cuenta que la eliminación se maneja como una baja lógica. Las contraseñas se derivan con PBKDF2 y una sal aleatoria en segundo plano y no se guardan en texto plano, por lo que el ingreso muestra un estado de espera mientras termina la validación.

La biometría guarda el identificador de la sesión cifrado con una clave del Keystore que requiere autenticación y al cerrar la sesión se desactiva este acceso. Para una versión comercial será necesario validar los permisos desde el servidor y no depender únicamente de los controles que se muestran en la pantalla.

Interfaz de acceso ejecutada

En la siguiente imagen se muestra la apertura del APK instalado en un emulador Android 11 API 30 y se puede observar que la pantalla conserva los colores del negocio, junto con las opciones para ingresar con correo y contraseña, utilizar el acceso biométrico, crear una cuenta y recuperar el acceso de prueba.



El contenido de esta pantalla se puede desplazar para consultar las cuentas de demostración y continuar con la revisión del aplicativo.

Validación y comprobaciones

Para revisar la compilación se ejecutaron assembleDebug, testDebugUnitTest y lintDebug y el resultado fue BUILD SUCCESSFUL, teniendo en cuenta que el APK está firmado para depuración y declara API mínima 26 y SDK objetivo 35. La revisión de Lint registró 0 errores y 76 advertencias relacionadas principalmente con textos y estilo, por lo que este resultado no significa que el proyecto esté libre de advertencias.

Prueba automatizada	Resultado esperado
Totales con precios del menú	3 cachapas de $8.000 y una de $15.000 suman $39.000.
Formato de moneda	25000 se presenta como $25.000.
Cantidades e inventario	Se rechazan cero, negativos, más de 99 y cantidades superiores al stock.
Carrito vacío y precios	No se confirma un carrito vacío ni un producto con precio cero.
Estados del pedido	Se impide pasar de Recibido directamente a Entregado.
Roles y correo	El comprador no gestiona el catálogo y se rechaza un correo inválido.

En JUnit se ejecutaron 9 pruebas con 0 fallos, 0 errores y 0 pruebas omitidas y los casos se encuentran en RulesTest.kt, de manera que estas comprobaciones se pueden repetir al revisar el código del proyecto.

Pruebas de interfaz

Durante la revisión se instaló y abrió el APK en un emulador Android 11 API 30 y se detectaron bloqueos al preparar las cuentas y validar las contraseñas, por lo que estas operaciones se trasladaron a segundo plano. Después de este ajuste se volvió a compilar el proyecto y se ejecutaron nuevamente las pruebas.

La validación completa de los tres roles y de la biometría debe repetirse en un teléfono o en un emulador estable, teniendo en cuenta que las capturas de este documento muestran la apertura de la interfaz y no demuestran por sí solas que ya se hayan comprobado todos los recorridos.

Verificación adicional antes de entregar

Para revisar la huella se necesita un dispositivo con biometría fuerte y un registro configurado y adicionalmente se debe comprobar que el ingreso con contraseña siga disponible cuando no se pueda utilizar esa opción. Antes de presentar el proyecto también conviene repetir el recorrido en el teléfono que se va a utilizar y revisar el tamaño de la letra y el teclado, dado que las metas de concurrencia, disponibilidad y tiempos del backend del documento anterior no forman parte de esta validación local.

Ejecución y generación del APK

La carpeta CachapasAndroid contiene el proyecto que se debe abrir en Android Studio y allí se encuentran el código, los recursos, el manifiesto, la configuración de Gradle y el wrapper. No se incluyeron las rutas privadas de un SDK ni archivos de configuración de otro equipo y de esta manera la aplicación puede prepararse en el computador que se utilizará para la entrega.

1. Para comenzar se instala Android Studio con un JDK compatible y el SDK Android 35, teniendo en cuenta que el proyecto utiliza Android Gradle Plugin 8.7.3, Gradle 8.9 y compatibilidad de código con Java 17.

2. Después se extrae el ZIP y se elige Open en Android Studio para seleccionar la carpeta CachapasAndroid que contiene settings.gradle.kts y se espera la sincronización de Gradle. La primera ejecución necesita conexión a Internet para descargar las dependencias.

3. Cuando Android Studio solicite componentes del SDK se acepta su instalación y se conserva la estructura del proyecto entregado, por lo que no es necesario crear una nueva plantilla de aplicación sobre esa carpeta.

4. Para ejecutar se crea un emulador o se conecta un teléfono con depuración USB y se pulsa Run, teniendo en cuenta que la aplicación admite Android 8.0 API 26 o una versión superior.

5. Para compilar desde Windows se abre una terminal en CachapasAndroid y se ejecuta gradlew.bat assembleDebug y en macOS o Linux se utiliza ./gradlew assembleDebug. El archivo generado se encuentra en app/build/outputs/apk/debug/app-debug.apk.

6. Las comprobaciones se pueden repetir con gradlew.bat testDebugUnitTest lintDebug en Windows o con ./gradlew testDebugUnitTest lintDebug en macOS o Linux y los informes se guardan en app/build/reports.

7. Si se necesita reiniciar el catálogo se borran los datos de la aplicación desde los ajustes de Android o se desinstala y se vuelve a instalar, teniendo en cuenta que esta acción elimina las cuentas y los pedidos creados en ese dispositivo.

Cuentas de demostración

Rol	Correo
Comprador	comprador@cachapas.demo
Vendedor	vendedor@cachapas.demo
Administrador	admin@cachapas.demo

La contraseña inicial de las tres cuentas es Cachapas2026! y los botones Explorar como permiten completar las credenciales, aunque después se debe pulsar Ingresar para continuar. Si el administrador modifica una cuenta se utiliza la contraseña actualizada y el APK de depuración incluido se mantiene como un archivo para la validación académica.

Publicación y materiales de entrega

El repositorio debe contener el proyecto completo y tener acceso público para que el docente pueda realizar la validación, teniendo en cuenta que los archivos temporales, las cachés, las compilaciones y local.properties se excluyen mediante .gitignore. El menú original sí permanece incluido porque forma parte de los recursos utilizados en la aplicación.

Crear el repositorio en GitHub

1. Para publicar se ingresa a GitHub con la cuenta Javier9905 y se crea un repositorio público llamado EstoEsCachapas-Android, dejándolo vacío al momento de su creación.

2. En la carpeta CachapasAndroid se abre una terminal y se ejecutan git init, git add . y git commit -m "Actividad 4 interfaces Android de Cachapas" y después se utiliza git branch -M main para definir la rama principal.

3. El repositorio se vincula con git remote add origin https://github.com/Javier9905/EstoEsCachapas-Android.git y se sube el código con git push -u origin main. Si el computador no tiene una sesión autorizada, Git solicitará iniciar sesión para completar la publicación.

4. Después de publicar se abre el enlace en una ventana privada para comprobar que pueda consultarse sin iniciar sesión y se copia esa dirección para adjuntarla en la entrega del aula.

5. Finalmente se adjuntan el ZIP con el código, la documentación técnica y el enlace del video de máximo cinco minutos y se revisa que el video también tenga permisos de visualización.

El nombre indicado para el repositorio corresponde a la publicación que se debe realizar y no representa un enlace que ya esté creado, por lo que la aplicación puede compilarse y revisarse antes de subir el código a GitHub.

Estado de las integraciones propuestas

Función	Estado en esta entrega
Pantallas y navegación de tres roles	Implementadas con XML y Kotlin.
CRUD, carrito e historial	Implementados con almacenamiento local.
Biometría	Integrada con AndroidX Biometric y Keystore; depende del dispositivo.
Pago y recuperación de contraseña	Demostraciones locales identificadas en pantalla.
Firebase, Wompi, cámara y ubicación	Trabajo posterior. No se presentan como integraciones operativas.

Guion del video técnico

Para la grabación se propone una duración de cuatro a cinco minutos y se deben mostrar Android Studio y el emulador, teniendo preparado un pedido aprobado y las cuentas de demostración para seguir el recorrido.

Inicio y alcance de 0 a 30 segundos

Hola, soy Javier Andrés Barrera Cárdenas. Lo que presento en esta actividad es el desarrollo de las interfaces Android de Esto Es Cachapas y Algo Más y para ello adapté la propuesta de TecnoConecta a mi proyecto de comidas, conservando los tres roles solicitados. La aplicación utiliza Kotlin con layouts XML y en esta versión los datos se guardan en el dispositivo y los pagos son de prueba.

Comprador de 30 segundos a 1 minuto 25 segundos

Como primera parte ingreso con el rol de comprador y desde el inicio puedo consultar los productos destacados y abrir el menú, por ejemplo: cachapas, hamburguesas, pizzas y los demás productos con los precios del negocio. El catálogo permite buscar por nombre y filtrar por categoría y al abrir una cachapa puedo elegir la cantidad, agregarla al carrito, cambiar las unidades o eliminarla antes de confirmar el contacto de recogida. La pasarela es una simulación y permite elegir un resultado aprobado, pendiente o rechazado sin solicitar datos bancarios, teniendo en cuenta que un resultado aprobado crea el pedido y descuenta la disponibilidad.

Vendedor y administrador de 1 minuto 25 a 2 minutos 20 segundos

Ahora ingreso como vendedor y desde productos puedo crear, consultar, editar y eliminar una publicación mediante una baja lógica, por lo que el formulario revisa el nombre, el precio y las unidades antes de guardar. En pedidos recibidos consulto el pedido creado y puedo pasarlo a preparación, listo para recoger y entregado y después ingreso como administrador para gestionar usuarios, modificar sus datos o su rol y desactivar una cuenta. El reporte muestra las ventas de prueba por producto y solo suma los pedidos con resultado aprobado.

Layouts y recursos de 2 minutos 20 a 3 minutos 30 segundos

En Android Studio se encuentra activity_main.xml y este LinearLayout organiza el toolbar, el contenedor de fragmentos y la navegación inferior, teniendo en cuenta que cada destino tiene su propio archivo XML. Por ejemplo, fragment_catalog.xml utiliza ScrollView, un campo de búsqueda y un ChipGroup para las categorías y las tarjetas se reutilizan desde item_card.xml. En drawable se encuentran los iconos y los fondos con esquinas y gradientes y en values se definieron colors.xml, dimens.xml, strings.xml y styles.xml, de manera que los colores del negocio se mantienen en los componentes que los utilizan.

Lógica y funciones nativas de 3 minutos 30 a 4 minutos 20 segundos

ScreenFragment relaciona las vistas con sus acciones y Store organiza la persistencia, los permisos por rol y los pedidos, mientras que las contraseñas se procesan en segundo plano para conservar disponible la interfaz. Domain contiene las reglas de cantidades y totales y cada pedido guarda una copia del precio para conservar el historial cuando después cambie un producto. La biometría se activa desde Perfil y utiliza BiometricGate, el diálogo del sistema y una clave en Android Keystore, por lo que la aplicación no guarda huellas y permite seguir ingresando con contraseña cuando el equipo no tenga esa función. La recuperación de acceso también es una demostración local y muestra su código en pantalla.

Validación y cierre de 4 minutos 20 a 4 minutos 50 segundos

Para terminar muestro las pruebas y la generación del APK, teniendo en cuenta que las comprobaciones revisan totales, disponibilidad, moneda, roles y estados del pedido y que el wrapper de Gradle permite repetir la compilación. Con esta versión dejo organizadas las interfaces de la actividad y para continuar con una operación comercial faltaría conectar el backend, validar pagos reales y confirmar los datos de entrega del negocio. Gracias.

Privacidad y adaptación normativa

La protección de datos y la información de la compra se mantienen como parte del proyecto al pasar de un catálogo de tecnología a uno de alimentos y la Ley 1581 de 2012 orienta el tratamiento de los datos personales, teniendo en cuenta que en esta versión se utilizan datos locales de prueba y ese alcance se explica dentro de la aplicación. Antes de trabajar con clientes reales será necesario definir el responsable del tratamiento, las finalidades, las autorizaciones y los canales de atención que correspondan.

La Ley 1480 de 2011 sirve como referencia para la información del producto y las ventas por medios electrónicos y por ello la ficha comercial deberá completar los ingredientes, la disponibilidad, el total y las condiciones de entrega. El artículo 47 contempla excepciones al retracto para bienes perecederos o que puedan deteriorarse rápidamente y teniendo en cuenta este cambio de productos, no se trasladó una devolución general de accesorios a los alimentos preparados, aunque esto no reemplaza la atención de los reclamos por calidad o incumplimiento.

En el prototipo se diferencia la cancelación de un intento sin confirmar y la de un pedido aprobado, puesto que el reintegro que se muestra es simulado y conserva su registro. Para integrar un pago real se deberá verificar la transacción en el servidor y ejecutar el trámite que corresponda a cada caso.

Referencias

Android Developers. Android Gradle Plugin 8.7.0 y actualizaciones 8.7.3. https://developer.android.com/build/releases/agp-8-7-0-release-notes

Android Developers. Show a biometric authentication dialog. https://developer.android.com/identity/sign-in/biometric-auth

Android Developers. Save UI states. https://developer.android.com/topic/libraries/architecture/saving-states

Javier Andrés Barrera Cárdenas. Documento técnico de TecnoConecta para Android. 12 de octubre de 2026. Documento base del proyecto.

Esto Es Cachapas y Algo Más. Menú digital suministrado para esta actividad. Fuente de productos, precios e identidad visual.

Congreso de Colombia. Ley 1480 de 2011, artículos 47 y 50. Texto compilado en el Normograma del Ministerio de Relaciones Exteriores. https://www.cancilleria.gov.co/normograma/compilacion/docs/pdf/ley_1480_2011.pdf

Congreso de Colombia. Ley 1581 de 2012. Gestor Normativo de Función Pública. https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=49981

Las fuentes web se consultaron el 10 de noviembre de 2026 y las referencias normativas se utilizaron para orientar el diseño académico, teniendo en cuenta que deberán revisarse para la operación comercial concreta.